

Conceptos avanzados

22. Accesibilidad

- Atributos de “aria”

— **aria-hidden** : Atributo de tipo booleano que indica si un elemento y todos sus descendientes deben ser omitidos por las tecnologías de asistencia, como los lectores de pantalla, independientemente de su visibilidad visual en la interfaz, pero a diferencia de por ejemplo el atributo HTML hidden o la propiedad CSS display: none, este atributo no oculta el elemento de la pantalla para usuarios videntes.

Se utiliza principalmente para ocultar contenido puramente decorativo, iconos que ya tienen una etiqueta de texto adyacente o elementos redundantes, asegurando que la experiencia de navegación asistida sea fluida y carezca de ruido informativo innecesario.

— **aria-expanded** : Indica si el elemento que controla un contenedor oculto (por ejemplo un menú, un acordeón o un submenú) se encuentra actualmente expandido (true) o colapsado (false) siendo fundamental para informar al usuario de tecnologías de asistencia sobre el resultado de una acción de activación antes de que navegue hacia el nuevo contenido.

Se implementa en botones de control y disparadores de interactividad, permitiendo que el software de lectura de pantalla anuncie el cambio de estado dinámico de la interfaz tras una interacción del usuario.

— **aria-controls** : Establece una relación lógica entre el elemento actual y otro elemento de la interfaz que es controlado o modificado por él, su valor debe coincidir con el identificador (id) del elemento destino y se emplea sistemáticamente para ayudar a los usuarios de lectores de pantalla a comprender qué parte de la página está cambiando al interactuar con un control específico, por ejemplo vinculando un botón de "Filtros" con el contenedor que muestra los resultados, permitiendo una navegación más intuitiva entre el controlador y su efecto.

— **aria-labelledby** : Define una etiqueta descriptiva para un elemento mediante la referencia al identificador (id) de otro elemento presente en el DOM, a diferencia de aria-label, que contiene texto estático, aria-labelledby establece una relación dinámica con el contenido visible en pantalla.

Se utiliza cuando el nombre de un componente (como un cuadro de diálogo o una sección) ya está escrito visualmente en la interfaz (por ejemplo un título <h2>), evitando la redundancia de datos y garantizando que cualquier actualización en el texto del título se refleje automáticamente en la etiqueta del componente.

— **aria-describedby** : Vincula un elemento con uno o más elementos descriptivos mediante sus identificadores únicos (id) y se utiliza para proporcionar información contextual extendida, instrucciones detalladas o mensajes de error

que complementan la identidad del elemento, siendo fundamental en interfaces de formularios complejos para asociar campos de entrada con requisitos de validación, sugerencias de formato o advertencias dinámicas, garantizando que las tecnologías de asistencia comuniquen el contenido explicativo de manera automática cuando el usuario sitúa el foco sobre el componente relacionado.

— **aria-live** : Define cómo se deben comunicar a las tecnologías de asistencia los cambios realizados en una región de la página que se actualiza dinámicamente sin que el usuario mueva el foco, admite valores como polite (anuncia el cambio cuando el usuario termina su tarea actual) o assertive (interrumpe la lectura para anunciar el cambio inmediatamente).

Se utiliza en notificaciones de alerta, actualizaciones de resultados de búsqueda en tiempo real o contadores de tiempo, garantizando que los usuarios no pierdan información crítica que ocurre fuera de su área de enfoque inmediato.

— **aria-modal** : Atributo booleano que indica si el elemento sobre el cual se aplica (generalmente un diálogo o ventana emergente) constituye una ventana modal que restringe la interacción al contenido dentro de sí mismo, al establecerse en true informa a las tecnologías de asistencia que el contenido fuera de ese contenedor debe ser ignorado mientras la ventana esté activa.

Se implementa en cuadros de diálogo personalizados para asegurar que el lector de pantalla no "escape" hacia el contenido de fondo, forzando un flujo de navegación contenido y coherente con la lógica visual de la aplicación.

— **aria-current** : Indica que el elemento representa el ítem activo o actual dentro de un contenedor o conjunto de elementos relacionados, posee valores específicos según el contexto, como page (página actual en un menú), step (paso actual en un proceso) o date (fecha actual en un calendario).

Se emplea principalmente en sistemas de navegación y migas de pan (breadcrumbs) para resaltar semánticamente la posición exacta del usuario dentro de la estructura de la aplicación, permitiendo que los lectores de pantalla identifiquen visual y auditivamente el estado de selección activo.

— **aria-disabled** : Indica que un elemento es perceptible pero no es editable ni operativo en el momento actual, a diferencia del atributo HTML disabled, el elemento marcado con aria-disabled="true" puede seguir recibiendo el foco del teclado, lo cual es útil para proporcionar explicaciones de por qué el control está inhabilitado mediante un tooltip o descripción.

Se utiliza en interfaces donde se desea mantener la visibilidad y el orden de tabulación de un control (como un botón de "Enviar" que requiere completar el formulario primero), informando al usuario sobre su presencia sin permitir su ejecución.

- **Roles de "aria"**

— **dialog** : Identifica un elemento como una ventana de aplicación o cuadro de

diálogo que se superpone al contenido principal del documento, a diferencia de un documento estándar, un elemento con este rol establece un contexto de interacción específico donde se espera que el usuario realice una tarea o reciba información antes de retomar el flujo principal.

Se utiliza para marcar contenedores que funcionan como modales o ventanas emergentes personalizadas, permitiendo que las tecnologías de asistencia anuncien la apertura de una interfaz independiente y gestionen el foco del teclado de manera contenida dentro del componente.

— **alert** : Comunica mensajes importantes y de carácter urgente al usuario, cuando un elemento con este rol aparece o se actualiza en el DOM, el navegador instruye a las tecnologías de asistencia para que interrumpan cualquier lectura en curso y anuncien el contenido de inmediato.

Se implementa exclusivamente para notificaciones críticas, como errores de validación de formularios, alertas de pérdida de conexión o advertencias de sistema que requieren atención inmediata, garantizando que el usuario perciba la información sin importar su ubicación actual en la página.

— **status** : Proporciona información sobre el estado de un proceso o de la aplicación, pero que no son lo suficientemente críticos como para interrumpir al usuario, a diferencia de alert el contenido de un elemento status se anuncia de manera "polite" (educada), esperando a que el sintetizador de voz esté inactivo.

Se emplea para confirmar acciones exitosas (como "Documento guardado"), mostrar el progreso de una carga o indicar resultados de búsqueda actualizados, manteniendo al usuario informado sobre cambios dinámicos sin fragmentar su experiencia de navegación.

— **navigation** : Identifica una sección del documento que contiene enlaces para navegar por el sitio web o por la página actual, proporcionando un punto de referencia (landmark) que permite a los usuarios de tecnologías de asistencia saltar directamente a los menús de navegación principales sin tener que recorrer todo el contenido previo.

Se utiliza sobre elementos contenedores (como <div> o) cuando la etiqueta nativa <nav> no puede emplearse o cuando se requiere reforzar la semántica en estructuras complejas, facilitando la orientación espacial dentro de la arquitectura de la información.

— **main** : Designa el contenido central y único del documento, diferenciándolo de elementos repetitivos como encabezados, pies de página o barras laterales de navegación, solo debe existir una instancia de este rol visible por página (o por contexto de anidación superior) y se implementa para permitir que los usuarios de lectores de pantalla utilicen comandos de "salto al contenido" (skip links) de forma nativa, mejorando drásticamente la eficiencia en la navegación al ignorar bloques de información redundantes que aparecen en todas las páginas del sitio.

— **tablist** : Define un elemento como el contenedor principal de un conjunto de

controles de pestañas, estableciendo la relación jerárquica necesaria para que el navegador comprenda que los elementos hijos forman parte de un sistema de navegación por paneles.

Se utiliza en el desarrollo de componentes de tipo Tabs para agrupar múltiples elementos con `role="tab"`, permitiendo que el software de asistencia anuncie al usuario cuántas opciones hay disponibles y cuál es la posición actual dentro del grupo de selección.

— **tab** : Representa un elemento interactivo que al ser activado, revela un panel de contenido asociado y oculta los demás, para esto debe estar contenido obligatoriamente dentro de un elemento con `role="tablist"`.

Se emplea para identificar cada una de las "pestañas" individuales, informando al usuario sobre su estado de selección (generalmente mediante `aria-selected`) y vinculándose con su respectivo contenedor de información, lo que facilita una navegación estructurada en interfaces compactas.

— **tabpanel** : Designa el panel que alberga el contenido asociado a una pestaña específica (`role="tab"`), este elemento debe estar vinculado mediante el atributo `aria-labelledby` al control que lo activa y se utiliza para estructurar la información que se muestra u oculta dinámicamente en una interfaz de pestañas, asegurando que cuando un usuario activa una opción, el lector de pantalla identifique correctamente el bloque de contenido que acaba de volverse relevante y accesible.

— **switch** : Representa un tipo de botón que permite al usuario alternar entre dos estados opuestos, de forma similar a un interruptor físico (encendido/apagado), a diferencia de un checkbox tradicional, el switch implica una acción inmediata tras su activación.

Se utiliza en paneles de configuración y ajustes de usuario donde se requiere una respuesta instantánea del sistema, proporcionando una semántica clara sobre la naturaleza binaria y el estado actual (activado o desactivado) de la función controlada.

— **grid** : Define un contenedor como una cuadrícula o tabla interactiva que permite la navegación por filas y columnas mediante el teclado, a diferencia de una tabla estática (`<table>`), el rol grid implica que el usuario puede interactuar con las celdas, realizar selecciones o editar datos.

Se implementa en aplicaciones de hojas de cálculo, calendarios complejos o tablas de datos dinámicas donde se requiere un control preciso sobre el enfoque de cada celda y la gestión de grandes volúmenes de información estructurada.

— **listbox** : Identifica un elemento que permite al usuario seleccionar una o más opciones de una lista estática de elementos, siendo la contraparte semántica del elemento nativo `<select>` pero aplicado a estructuras personalizadas del DOM.

Se utiliza cuando se desarrollan menús desplegables o selectores personalizados que requieren un diseño visual específico no soportado por los

controles nativos del sistema operativo, permitiendo que el navegador gestione la selección de sus elementos hijos (role="option").

— **option** : Representa una opción individual seleccionable dentro de un contenedor con role="listbox", role="grid" o role="tree" que debe informar siempre de su estado de selección actual. Se emplea para marcar cada ítem dentro de un menú de selección o lista desplegable, asegurando que el usuario reciba confirmación auditiva de qué opción está resaltada o seleccionada antes de confirmar su elección definitiva.

23. Rendimientos y carga de recursos

• Carga de scripts

— **defer** : Atributo booleano aplicable a scripts externos que instruye al navegador para descargar el archivo de forma asíncrona mientras continúa el procesamiento del análisis sintáctico (parsing) del HTML, a diferencia de los scripts estándar, la ejecución del código se pospone estrictamente hasta que el documento ha sido completamente interpretado, pero antes de que se dispare el evento DOMContentLoaded.

Se utiliza prioritariamente cuando el script depende de la existencia de elementos específicos en el DOM o cuando se requiere mantener un orden de ejecución predecible entre múltiples archivos, optimizando el tiempo de renderizado inicial sin bloquear la visualización de la página.

— **async** : Atributo booleano que permite que un script externo se descargue en segundo plano sin interrumpir el análisis del documento HTML, sin embargo a diferencia de defer, el navegador detendrá el análisis del HTML para ejecutar el script en el instante exacto en que la descarga finalice, reanudando el procesamiento posterior.

Se emplea fundamentalmente para scripts que son independientes del resto de la lógica de la aplicación y que no requieren manipular el DOM de forma crítica en la carga inicial, como servicios de analítica, plataformas de publicidad externa o rastreadores de telemetría, donde la prioridad es la carga rápida sin garantizar un orden de ejecución específico.

• Optimización de carga

— **loading="lazy"** : Instruye al navegador para posponer la descarga de recursos (como imágenes o iframes) hasta que se aproximan a una distancia calculada del área de visualización del usuario (viewport), el motor de renderizado reservará el espacio del elemento pero no iniciará la transferencia de red hasta que la probabilidad de interacción o visibilidad sea alta.

Se utiliza sistemáticamente para reducir el tiempo de carga inicial y el consumo de ancho de banda en páginas con alto contenido visual, mejorando métricas críticas de rendimiento como el Largest Contentful Paint (LCP) al priorizar únicamente los recursos críticos del primer segmento de la página.

— **loading="eager"** : Indica al navegador que el recurso debe descargarse y

procesarse de forma inmediata en cuanto se encuentre en el documento HTML, independientemente de su posición relativa al viewport, es el comportamiento por defecto de los navegadores, pero su declaración explícita sobrescribe cualquier configuración de carga diferida heredada o global. Se emplea exclusivamente para elementos críticos de la interfaz que deben estar disponibles instantáneamente, como logotipos de cabecera, imágenes principales de un artículo (hero images) o componentes de navegación esenciales que el usuario debe visualizar sin retardo al cargar la aplicación.

— **decoding="async"** : Permite al navegador realizar la decodificación de los datos de imagen (el paso de formato comprimido a mapa de bits en memoria) en un hilo de ejecución paralelo, en lugar de hacerlo en el hilo principal de la interfaz, evitando así que el procesamiento de imágenes pesadas bloquee la ejecución de JavaScript o la respuesta del usuario a eventos táctiles o de teclado. Se implementa en aplicaciones con alta densidad de gráficos para garantizar que el desplazamiento (scrolling) y las animaciones permanezcan fluidas, aceptando un ligero retraso en la aparición de la imagen a cambio de mantener la estabilidad del rendimiento de la interfaz.

— **fetchpriority** : Permite al desarrollador sugerir al navegador la importancia relativa de un recurso durante la fase de descarga de red, admitiendo los valores high (prioridad alta), low (prioridad baja) o auto (prioridad asignada por el navegador). Se utiliza para optimizar la secuencia de carga de activos, por ejemplo asignando high a la imagen principal de un producto para que se descargue antes que otros scripts menos críticos, o low a imágenes que se encuentran fuera de pantalla o carruseles secundarios, permitiendo un control granular sobre el uso del ancho de banda y la velocidad percibida de la aplicación.

- **Relaciones de red**

— **preconnect** : Instruye al navegador para iniciar las etapas de resolución de DNS, establecimiento de conexión TCP y negociación TLS hacia un origen externo antes de que se realice una solicitud formal de recursos, permitiendo que el agente de usuario complete el intercambio de protocolos de red en segundo plano, eliminando la latencia de conexión en el momento en que el documento requiere descargar un activo específico del dominio indicado. Se utiliza para agilizar el acceso a servidores de terceros, como proveedores de tipografías, servicios de análisis o redes de entrega de contenidos (CDN).

— **dns-prefetch** : Indica al navegador la necesidad de resolver la dirección IP correspondiente a un nombre de dominio específico de forma proactiva, el proceso se limita exclusivamente a la consulta al servidor DNS para obtener la dirección numérica del servidor de destino y se implementa en el desarrollo de interfaces para reducir el tiempo de espera en la carga de recursos secundarios o enlaces externos que el usuario probablemente activará, permitiendo que la resolución de nombres se complete antes de que se inicie

cualquier transferencia de datos.

— **modulepreload** : Optimiza la adquisición y el procesamiento de módulos de JavaScript (ES modules), ordenando al navegador descargar el archivo, analizar su sintaxis y compilarlo directamente en el mapa de módulos de la memoria del sistema.

Se emplea en arquitecturas de software basadas en módulos para garantizar que el código esté listo para su ejecución inmediata, procesando el archivo de forma anticipada para evitar retardos durante la fase de importación y ejecución de la lógica de la aplicación.

- **Preload**

— **link rel="preload"** : Ordena al navegador solicitar un recurso con una prioridad alta debido a su importancia crítica para el renderizado de la página actual, obligando al motor de descarga a iniciar la transferencia del activo de forma inmediata, incluso antes de que el analizador de HTML encuentre la referencia natural al recurso en el cuerpo del documento.

Se utiliza para garantizar que elementos esenciales, como hojas de estilo principales o scripts de configuración inicial, estén disponibles en la memoria del navegador lo antes posible, reduciendo el tiempo de bloqueo visual durante la carga.

— **as="font"** : Utilizado dentro de la directiva preload para indicar al navegador la naturaleza técnica del archivo que se va a descargar, al definir el tipo como una fuente tipográfica, el navegador aplica las políticas de seguridad adecuadas, como la gestión de intercambio de recursos de origen cruzado (CORS), y asigna la prioridad de red correspondiente a los activos binarios de texto.

Se emplea para asegurar que los archivos de tipografía (como WOFF2) se descarguen antes de que el navegador intente renderizar el texto, evitando efectos visuales no deseados como el parpadeo de fuentes no cargadas.

— **imagesrcset** : Utilizado dentro de un elemento `<link rel="preload">` para proporcionar una lista de diferentes versiones de una misma imagen, cada una con dimensiones o resoluciones específicas, permitiendo que el navegador evalúe las características del dispositivo del usuario (como el ancho de pantalla y la densidad de píxeles) para seleccionar y precargar únicamente la versión más adecuada del archivo.

Se implementa en estrategias de diseño responsivo para optimizar el rendimiento, asegurando que se descargue por adelantado la imagen óptima para el contexto visual del usuario sin desperdiciar ancho de banda.

— **imagesizes** : Acompaña a la precarga de imágenes adaptativas para informar al navegador sobre el tamaño final que ocupará la imagen en el diseño de la página antes de que se procesen las reglas de CSS, proporcionando una serie de condiciones de medios y valores de ancho que ayudan al motor de renderizado a realizar el cálculo preciso de qué recurso del imagesrcset debe

ser descargado con prioridad.

Se utiliza fundamentalmente para asistir al navegador en la toma de decisiones de carga de recursos gráficos en interfaces complejas donde el tamaño de la imagen varía significativamente según el dispositivo.

24. Seguridad y privacidad

• Seguridad de recursos

— **integrity** : Permite al navegador verificar que un recurso externo (como un archivo JavaScript o CSS de un CDN) no ha sido modificado de forma maliciosa, conteniendo una cadena de caracteres que representa un resumen criptográfico (hash) del contenido esperado del archivo, generalmente utilizando algoritmos como SHA-256, SHA-384 o SHA-512, en resumen el navegador descarga el recurso, calcula su propio hash y lo compara con el valor del atributo, si no coinciden bloquea la ejecución del archivo para proteger la aplicación contra ataques de inyección de código o compromiso de servidores de terceros.

— **nonce** : Atributo de seguridad utilizado en el marco de las Políticas de Seguridad de Contenido (CSP) para autorizar la ejecución de scripts o la aplicación de estilos específicos que están incrustados directamente en el HTML (inline), consistiendo en una cadena criptográfica generada aleatoriamente por el servidor en cada solicitud de página, la cual debe coincidir exactamente con el valor permitido en la cabecera HTTP Content-Security-Policy.

Se implementa para prevenir ataques de Cross-Site Scripting (XSS), asegurando que solo se ejecuten los bloques de código que el servidor ha marcado explícitamente como legítimos en esa sesión específica.

— **crossorigin** : Define el modo de solicitud CORS (Cross-Origin Resource Sharing) al adquirir recursos de un dominio distinto al de la página principal. Admite valores como anonymous (solicita el recurso sin enviar credenciales como cookies o certificados) o use-credentials (incluye las credenciales del usuario en la petición).
Se emplea para permitir que los navegadores accedan a datos de recursos externos que, por seguridad, están restringidos por la política de mismo origen, siendo indispensable para el correcto funcionamiento de elementos como <canvas>, fuentes web o scripts que requieren lectura de errores detallada.

— **referrerpolicy** : Determina cuánta información sobre la procedencia de la solicitud (la URL de origen) debe incluirse en la cabecera Referer al realizar una petición de red, admite diversas directivas como no-referrer, origin, strict-origin-when-cross-origin, entre otras, permitiendo filtrar u ocultar datos sensibles de la URL de la página actual.
Se utiliza para proteger la privacidad de los usuarios y la seguridad de la estructura de la aplicación, evitando que parámetros privados o rutas internas sean filtrados a servidores externos durante la carga de imágenes, scripts o redirecciones de enlaces.

- **Seguridad en iframes**

|
|— **sandbox** : Aplicado al elemento <iframe> que habilita un conjunto de restricciones extra sobre el contenido anidado, tratándolo como si proviniera de un origen único y forzando una política de privilegios mínimos, al declararse este atributo el navegador deshabilita por defecto la ejecución de scripts, el envío de formularios, la apertura de ventanas emergentes y el acceso a las APIs de almacenamiento local, a menos que se concedan permisos específicos mediante valores como allow-scripts, allow-forms o allow-same-origin.

Se utiliza fundamentalmente para mitigar ataques de Cross-Site Scripting (XSS) y clickjacking al renderizar contenido de terceros o publicidad externa, garantizando que el código incrustado no pueda interferir con la lógica o los datos de la aplicación anfitriona.

- **Content Security Policy**

|
|— **meta http-equiv="Content-Security-Policy"** : Declara y aplica una política de seguridad directamente desde el documento HTML, emulando el comportamiento de una cabecera de respuesta HTTP, instruyendo al navegador sobre qué orígenes de recursos (scripts, estilos, imágenes o fuentes) se consideran legítimos y tienen permiso para ser cargados o ejecutados en el contexto de la página actual.

|
|
| Se implementa como una capa de defensa fundamental cuando el desarrollador no tiene acceso a la configuración de las cabeceras del servidor web, permitiendo restringir de forma granular las conexiones salientes y el contenido entrante para prevenir la ejecución de código no autorizado.

|
|— **CSP conceptual** : Detecta y mitiga tipos específicos de ataques, incluyendo el Cross-Site Scripting (XSS) y las inyecciones de datos, su funcionamiento se basa en definir una lista blanca de fuentes de contenido permitidas, bloqueando por defecto cualquier recurso que no cumpla con las directivas establecidas, como scripts integrados (inline scripts) o estilos no firmados. Se utiliza en el desarrollo de aplicaciones web robustas para reducir significativamente la superficie de ataque, delegando en el navegador la responsabilidad de verificar la integridad y el origen de cada activo antes de su procesamiento, garantizando así que solo se ejecute lógica de negocio verificada.

25. HTML como plataforma

- **Componentización**

|
|— **template** : Actúa como un contenedor para almacenar fragmentos de HTML que no deben ser renderizados ni procesados durante la carga inicial del documento, el contenido en su interior se mantiene en un estado inerte, lo que significa que los scripts no se ejecutan, las imágenes no se descargan y los estilos no se aplican hasta que el contenido es instanciado dinámicamente

mediante JavaScript.

Se utiliza fundamentalmente para definir estructuras de interfaz repetitivas o componentes personalizados que requieren ser clonados múltiples veces, optimizando el rendimiento al evitar el análisis redundante de estructuras que no son visibles de inmediato.

— **slot** : Utilizado dentro del Shadow DOM que funciona como un punto de inserción para contenido externo proporcionado por el documento principal, permitiendo la composición de componentes al definir dónde debe renderizarse el marcado que el usuario inserta entre las etiquetas de un elemento personalizado.

Se emplea para crear componentes flexibles y reutilizables, como tarjetas o modales, donde la estructura lógica y el estilo están definidos internamente, pero el contenido textual o gráfico es inyectado desde el exterior, manteniendo una separación clara entre la estructura del componente y sus datos.

— **Custom Elements (concepto)** : Permite a los desarrolladores definir sus propias etiquetas HTML con comportamientos y ciclos de vida personalizados, mediante el registro de una clase de JavaScript asociada a un nombre de etiqueta (que debe contener obligatoriamente un guion), el navegador reconoce el nuevo elemento y le otorga capacidades programáticas completas. Se utiliza para encapsular lógica de negocio y estructuras complejas en componentes autónomos y semánticos, facilitando la creación de bibliotecas de interfaz de usuario que son compatibles con cualquier entorno o framework sin dependencias externas.

— **Declarative Shadow DOM** : Define y adjuntar un Shadow DOM a un elemento directamente a través de HTML, sin necesidad de ejecutar scripts de inicialización en el cliente, utilizado con el elemento `<template>` junto con el atributo `shadowrootmode` para serializar la estructura interna del componente desde el servidor.

Se implementa para mejorar el rendimiento de la carga inicial y habilitar el renderizado en el lado del servidor (SSR) en componentes web, garantizando que el estilo y la estructura encapsulada estén disponibles para el navegador antes de que el motor de JavaScript comience su ejecución.

— **shadowrootmode="open"** : Permite que el árbol de elementos internos de un componente sea accesible y manipulable desde el documento principal a través de la propiedad `shadowRoot` de JavaScript, aunque el estilo permanece encapsulado y protegido de selectores CSS globales, este modo permite la inspección y la interacción programática externa con la estructura interna del componente.

Se utiliza en la mayoría de los casos de desarrollo de componentes web para permitir integraciones de herramientas de prueba, scripts de automatización y una depuración más sencilla durante el proceso de desarrollo.

— **shadowrootmode="closed"** : Establece un nivel de aislamiento estricto donde el árbol del Shadow DOM es completamente inaccesible para los scripts

externos del documento principal, al activarse, la propiedad `shadowRoot` del elemento devuelve `null`, impidiendo cualquier manipulación o inspección de la estructura interna por parte de bibliotecas o códigos ajenos al componente. Se emplea en escenarios específicos donde se requiere una protección total de la implementación interna, evitando que dependencias de terceros o scripts maliciosos interfieran con el estado o el marcado privado de un componente crítico de seguridad o de sistema.

- **Interoperabilidad**

- **view-transition-name** : Asigna un nombre único a un elemento que debe ser rastreado por la API de View Transitions durante un cambio de estado en el DOM o una navegación entre páginas, al definir este identificador el motor de renderizado del navegador captura una instantánea del elemento en su estado inicial y final, permitiendo generar animaciones fluidas y transformaciones automáticas (como desplazamientos, cambios de escala o desvanecimientos) entre ambas posiciones.

- Se utiliza fundamentalmente en el desarrollo de interfaces modernas para crear experiencias de usuario cinemáticas, asegurando que elementos persistentes, como imágenes de cabecera o avatares de perfil, se desplacen de forma coherente y visualmente integrada durante las transiciones de contenido sin necesidad de animaciones manuales complejas.

- **portal** : Elemento experimental de HTML diseñado para permitir la inserción de un contexto de navegación completo (una página web externa o interna) dentro de otro documento, de manera similar a un `<iframe>`, pero con la capacidad única de realizar una transición fluida hacia el contenido incrustado, a diferencia de los marcos tradicionales un `<portal>` permite que el usuario "entre" en el contenido mediante la API `activate()`, convirtiendo la página pre-renderizada en el documento principal sin recargar el navegador ni perder el estado de la aplicación.

- Se utiliza en el desarrollo de experiencias de usuario de alto rendimiento para implementar pre-visualizaciones interactivas de artículos, productos o páginas de destino que se activan instantáneamente al ser seleccionadas, eliminando los tiempos de espera perceptibles durante la navegación entre rutas o dominios.

26. Datos estructurados

- **Microdata**

- **itemscope** : Atributo booleano que indica al motor de procesamiento que el elemento HTML sobre el cual se aplica, junto con todos sus descendientes, constituye un objeto de datos específico o una entidad semántica, al declarar este atributo se crea un nuevo contexto de metadatos donde las propiedades definidas internamente se asocian exclusivamente a dicho grupo.

- Se utiliza para delimitar bloques de información estructurada dentro del documento, permitiendo que los motores de búsqueda identifiquen dónde comienza y termina la descripción de una entidad, como una persona, un

producto o un evento, sin confundirla con el resto del contenido de la página.

— **itemtype** : Especifica la categoría o el tipo de entidad que representa el objeto definido por un itemscope, su valor debe ser una URL válida que apunte a un vocabulario reconocido, siendo el estándar más común Schema.org (por ejemplo <https://schema.org>).

Se emplea para proporcionar un contexto semántico universal que permite a los sistemas de indexación comprender la naturaleza de los datos, facilitando la generación de fragmentos enriquecidos (rich snippets) en los resultados de búsqueda y mejorando la interoperabilidad de la información entre diferentes plataformas y servicios web.

— **itemprop** : Identifica una propiedad específica de una entidad dentro de un microdato, el valor asignado corresponde al nombre del campo según el esquema definido en itemtype (como name, author o price), y el contenido de la propiedad se extrae directamente del valor del elemento (texto, valor de un atributo src o contenido de un meta).

Se utiliza para etiquetar fragmentos granulares de información dentro de una estructura, permitiendo que los algoritmos de extracción de datos asocien valores concretos a los atributos del objeto semántico correspondiente.

— **itemid** : Proporciona un identificador único y persistente para una entidad, funcionando como una clave primaria global para el objeto de datos, a diferencia de un id de HTML que es local al documento, el itemid permite relacionar la misma entidad a través de múltiples páginas o sitios web mediante una URL o un código URN (Uniform Resource Name). Se implementa en arquitecturas de datos vinculados (Linked Data) para asegurar que diferentes descripciones del mismo objeto en distintas ubicaciones sean reconocidas como la misma instancia semántica, evitando la duplicidad de registros en bases de datos de conocimiento.

— **itemref** : Asocia propiedades a una entidad que no se encuentran físicamente dentro de su bloque itemscope, conteniendo una lista de identificadores (id) de otros elementos presentes en el documento que deben ser procesados como parte del objeto semántico principal.

Se utiliza para construir estructuras de datos complejas en casos donde el diseño visual o la jerarquía del DOM no permiten agrupar toda la información en un solo contenedor padre, garantizando que el modelo de datos final sea completo y coherente a pesar de la dispersión de los elementos en el código fuente.

- **Alternativa moderna**

— **JSON-LD** : Método de serialización estándar basado en JSON que permite estructurar datos semánticos y establecer relaciones entre entidades de forma independiente al marcado HTML, a diferencia de los microdatos incrustados en etiquetas, este formato se implementa dentro de un elemento `<script type="application/ld+json">`, lo que facilita la separación clara entre la

estructura visual del documento y su modelo de datos subyacente. Se utiliza como la solución preferente en el desarrollo web moderno para proporcionar metadatos críticos a los motores de búsqueda y sistemas de procesamiento de datos, permitiendo la definición de esquemas complejos de Schema.org sin afectar el rendimiento del renderizado ni la mantenibilidad del código fuente.

27. Modelo del documento (DOM)

- **Construcción del DOM**

— **Parsing del HTML** : Fase inicial del motor de renderizado donde el código fuente recibido en forma de flujo de bytes se transforma en una estructura de datos procesable, este proceso implica la decodificación de caracteres, la tokenización (división del código en etiquetas, atributos y contenido) y la validación sintáctica según las especificaciones del estándar.

Se ejecuta de forma continua para convertir el marcado textual en nodos de objetos que conformarán el árbol del DOM, permitiendo que el navegador comprenda la jerarquía y las relaciones de los elementos definidos en el archivo antes de proceder a la aplicación de estilos o la ejecución de scripts.

— **Parsing incremental** : Inicia el análisis y la construcción del DOM de forma simultánea a la descarga del archivo HTML, sin esperar a que el documento completo haya sido recibido por la red y a medida que llegan los paquetes de datos, el analizador procesa los fragmentos disponibles y los añade al árbol de nodos en tiempo real.

Se implementa para optimizar la velocidad percibida por el usuario, permitiendo que el contenido inicial de la página se visualice y sea interactivo mientras el resto del documento continúa descargándose en segundo plano.

— **Corrección automática de errores** : Mecanismo intrínseco de los navegadores modernos diseñado para garantizar la resiliencia del renderizado frente a un marcado HTML mal formado o inválido, en el que el analizador aplica reglas heurísticas para cerrar etiquetas omitidas, corregir anidamientos imposibles o gestionar elementos colocados fuera de su contexto legal (como una etiqueta <div> dentro de un <head>).

Se utiliza para asegurar que, a pesar de las deficiencias en el código fuente, la estructura final del DOM sea coherente y funcional, evitando el colapso de la interfaz y manteniendo la interoperabilidad entre diferentes implementaciones de motores web.

— **Tree construction** : Etapa final del análisis sintáctico donde los tokens generados se organizan en una estructura jerárquica de árbol conocida como el DOM Tree, en esta fase el motor establece las relaciones de parentesco (padres, hijos y hermanos) entre los nodos, creando un modelo de objetos donde la etiqueta <html> actúa como la raíz de toda la jerarquía.

Se emplea para proporcionar una representación viva de la interfaz en la memoria del sistema, la cual servirá como la base sobre la que interactuarán las hojas de estilo (CSSOM) y las APIs de JavaScript para la manipulación

dinámica del documento.

- **Modelo del DOM**

— **Nodos** : Representan los objetos individuales que conforman la estructura jerárquica de una página web, donde cada componente del marcado HTML es tratado como una instancia de una interfaz de programación, existiendo diversos tipos de nodos, siendo los más relevantes los de tipo elemento (etiquetas), texto (contenido alfanumérico), atributo y comentario. Se utilizan como la unidad mínima de manipulación en el desarrollo web, permitiendo que cada parte del documento posea propiedades y métodos específicos para su consulta, modificación o eliminación dinámica a través de lenguajes de programación.

— **Árbol DOM** : Representación lógica y organizada en forma de grafo arbóreo que el navegador genera a partir del análisis del código HTML, esta estructura comienza con un nodo raíz (típicamente el objeto document) y se ramifica en relaciones de parentesco de tipo padre-hijo y hermanos, reflejando fielmente la anidación del marcado original. Se emplea como el modelo vivo de la interfaz en la memoria del sistema, sirviendo de base para que el motor de renderizado calcule la disposición visual de los elementos y para que los scripts puedan navegar por la jerarquía del documento de manera eficiente.

— **API del DOM** : Constituye el conjunto de métodos, propiedades y eventos estandarizados que permiten a los lenguajes de programación (principalmente JavaScript) interactuar con la estructura del documento en tiempo real, esta interfaz expone funciones críticas como `querySelector()` para la localización de nodos, `addEventListener()` para la gestión de interactividad y `createElement()` para la generación de contenido dinámico. Se utiliza para transformar documentos estáticos en aplicaciones web interactivas, permitiendo la actualización de la interfaz de usuario sin necesidad de recargar la página completa y garantizando la interoperabilidad entre diferentes navegadores.

28. Ciclo de vida del documento

- **parsing** : Proceso mediante el cual el motor de renderizado del navegador transforma el flujo de bytes de un documento HTML en una estructura de datos lógica, esta fase se divide en dos pasos simultáneos: la tokenización, que fragmenta el código en etiquetas, atributos y texto, y el análisis de construcción, que procesa esos tokens para generar nodos en la memoria. Se ejecuta de manera continua a medida que llegan los datos de la red, permitiendo que el navegador comience a comprender la jerarquía del documento y a solicitar recursos externos (como hojas de estilo o imágenes) antes de que la descarga del archivo principal haya finalizado.
- **insertion modes** : Representan los estados internos del algoritmo de construcción del árbol que determinan cómo debe procesarse cada token entrante según el

contexto actual del documento, el analizador cambia entre diversos modos, como initial, before html, in body o in table, para aplicar reglas de validación específicas y manejar correctamente elementos mal colocados o etiquetas omitidas.

Se utilizan para garantizar la resiliencia del DOM, asegurando que el navegador sepa exactamente dónde insertar un nodo dentro de la jerarquía (por ejemplo, forzando la creación de un <tbody> si se encuentra un <tr> directamente dentro de una <table>) y manteniendo la coherencia estructural a pesar de errores de sintaxis.

- **tree construction** : Etapa del ciclo de vida donde los tokens validados por los modos de inserción se organizan en una estructura jerárquica de nodos vinculados, en este punto el motor establece las relaciones de parentesco (parent-child) y hermandad (siblings), transformando el marcado lineal en un grafo arbóreo donde el objeto document actúa como la raíz.
Esta fase es crítica porque genera el modelo vivo sobre el cual se aplicarán posteriormente las reglas de CSS y los scripts de JavaScript, sirviendo como la base estructural para el cálculo del diseño visual y la interactividad de la aplicación.
- **DOMContentLoaded** : Evento del ciclo de vida que se dispara en el objeto document una vez que el HTML ha sido completamente cargado y analizado, y el árbol de nodos está totalmente construido, en este punto el navegador ha procesado todos los scripts con el atributo defer, pero no necesariamente ha terminado de cargar recursos externos de gran volumen como imágenes, anuncios o hojas de estilo diferidas.
Se utiliza sistemáticamente para inicializar la lógica de la aplicación que requiere manipular elementos de la interfaz, garantizando que todos los selectores de nodos funcionen correctamente sin esperar a que la red complete la descarga de activos visuales no críticos.
- **load** : Evento final del ciclo de vida del documento que se dispara en el objeto window cuando la página y todos sus recursos dependientes han finalizado su carga con éxito, esto incluye archivos de imagen, marcos incrustados (iframes), scripts asíncronos y hojas de estilo.
Se emplea para ejecutar tareas que dependen de la disponibilidad total de los recursos o de sus dimensiones físicas finales, como el cálculo de tamaños de imágenes para galerías, la inicialización de animaciones complejas o el envío de métricas de rendimiento sobre el tiempo de carga total percibido por el usuario.